Document inventory database design

> User context
>
> Attachments:
>
> - Car Dealership Inventory System — Initial Planning and Database Design I want...: C:/Users/user/.codex/attachments/3f935e5d-5635-4b6b-8eef-99c5b9cdf621/Pasted text.txt

▶ 10 previous messages

The planning foundation is in place:

- README.md now covers the product scope, required routes, role policy, PostgreSQL setup, workflows, and an eleven-table Mermaid ER diagram.
- docs/database-design.md documents every table, relationship, constraint, index, deletion rule, and inventory transaction.
- AGENTS.md contains concise rules for future implementation. PROMPTS.md preserves your supplied prompt verbatim under a session entry.

I checked the design against the assignment PDF and available references, verified all eleven tables appear in both the diagram and data dictionary, confirmed the prompt bytes match the attachment, and ran `git diff --check`. No commit was made.

Assumptions to review: an active stock line is unique by case-insensitive make and model; users may log in before an email-verification workflow exists. The backend framework remains undecided. No application, migration, or tests were added or run; the PostgreSQL design has not yet been applied to a database.

Final Documentation Cleanup and Pre-Commit Review

Work in the existing repository: `Qusai7-04/dealership-inventory`.

First, inspect the current working tree and read `AGENTS.md`, `README.md`, `PROMPTS.md`, and the database design document. Read the original Incubyte assignment instructions available in the workspace as well.

1. Rewrite AGENTS.md using Ponytail principles

Use the upstream repository as the reference:

https://github.com/DietrichGebert/ponytail/blob/main/AGENTS.md

Adapt its “lazy senior developer” approach to this project. Do not merely keep the existing generic checklist, and do not copy unrelated project-specific rules blindly.

The instructions should emphasise:

- Understand the request and trace the affected code or documentation before making changes.
- Prefer the smallest complete solution; reuse existing patterns and avoid unnecessary abstractions, dependencies, files, and features.
- Be efficient without being careless about security, validation, data integrity, accessibility, or meaningful tests.
- Follow TDD for implementation work.
- Preserve PostgreSQL, the agreed inventory model, server-side authorization, transaction safety, and idempotency requirements.
- Keep documentation and commit history accurate.
- End task reports with what was skipped or not checked and any material risks.

Keep `AGENTS.md` compact, practical, and recognisably aligned with Ponytail's style. Do not install plugins, dependencies, or other tooling just to edit this file.

2. Audit README.md

Keep README as the current project guide, not an execution diary. Retain the useful project overview, required features versus extensions, role policy, PostgreSQL design summary, ER diagram, setup preparation, and `My AI Usage` section.

Verify every relative link against the actual repository files. Fix broken paths instead of leaving links to files that do not exist. Keep detailed database explanations in the database design document and link to it.

Remove any use of the prohibited internal milestone terminology from project documentation, including headings and link labels. Do not rename or duplicate files unnecessarily just to fix a link.

Do not claim PostgreSQL is installed, the application is running, migrations exist, or tests passed unless that has actually been verified. The current work is documentation and design; do not implement application features.

Keep the `My AI Usage` section truthful. It may state that Codex assisted with planning and database documentation, but leave personal reflection for me to complete.

3. Audit the database design

Cross-check the database design document against the README and the agreed eleven-table model:

`users`, `vehicle_categories`, `vehicles`, `vehicle_images`, `stock_receipts`, `favourites`, `carts`, `cart_items`, `purchases`, `purchase_items`, and `notifications`.

> Verify that the ER diagram, table descriptions, foreign keys, cardinalities, constraints, inventory rules, and transaction workflows agree. Do not add tables or features without a clear requirement. Record any unresolved design issue rather than silently guessing.
>
> Keep PostgreSQL installation and database creation instructions distinct from the future application schema and migration steps.

4. Audit PROMPTS.md

> Preserve all existing entries. Append this entire prompt verbatim under a clearly labelled session entry, using the current date.
>
> Do not summarise or rewrite the prompt. Do not invent a conversation transcript or public chat URL. Keep the file consistent with the assignment's requirement to preserve raw AI chat logs or public chat links; do not claim that prompt-only records are a complete transcript.

5. Verify the changes

> Inspect the repository's actual file paths and current Git status before editing. Modify only the files necessary for this cleanup.
>
> Then:
>
> - Check internal Markdown links and headings.
> - Compare the ER diagram against the written schema.
> - Check for accidental duplicated README sections or duplicated prompt entries.
> - Run appropriate documentation checks if existing tooling supports them; do not install a new dependency just for this.
> - Inspect the final diff and report exactly what changed and which checks actually ran.
>
> Do not commit, push, install PostgreSQL, or implement the backend/frontend. I will review and commit the changes manually.
>
> At the end, provide a concise summary of the changes, the checks performed, anything skipped or not checked, and any remaining risks.

▶ 6 previous messages

The documentation cleanup is ready for review.

- AGENTS.md now uses a compact, project-specific version of Ponytail's "lazy senior developer" approach.
- README.md no longer links to an ignored reference file. Its AI section accurately identifies the prompt records as incomplete for submission.
- docs/database-design.md records the remaining decisions. PROMPTS.md retains the earlier entry and contains this prompt under a dated session heading.

I checked local links and the heading anchor, all eleven diagram tables and their foreign-key relationships, duplicate headings and session entries, prohibited terminology in project documentation, and `git diff --check`. I inspected the final diff. The pre-existing `.gitignore` change was left untouched; no commit or push was made.

Not checked: PostgreSQL execution or application tests; neither a schema nor an application exists yet. **Remaining risk:** `docs/database-design.md` is untracked and must be included in your commit, and prompt-only records still need raw chat logs or public links before assignment submission.